Exercise 5: Claude Without You Watching

Session: 5 — Headless & Automation
Duration: 35 minutes, plus a 4-minute closing round
Project: The same BookStore API, and the `headless` folder in it.

Goal

You run Claude without a chat, and you check if you can trust the result. You:

1. Run one prompt headless, twice, and see what loads.
2. Build a review gate, and watch it pass one commit and block another.
3. Let a second run fix what the gate found, and check the fix yourself.
4. Write the prompt for a morning brief, and test it on two nights.

Start here

- Open a terminal in the `bookstore` folder you cloned. Run every command there.
- Run `uv --version`. uv runs the scripts in `headless`. No version? Ask the trainer.
- Training mode is on: `CLAUDE.md` makes Claude ask you a question first. Never edit `CLAUDE.md`. To get a direct answer in a chat, type *just tell me*.
- Write notes on paper or in a file outside the project.
- The trainer tells you when to start each task.

Tasks

1. One prompt, two runs

This task gives you two answers to one question. You write down why they differ.

1. **Predict.** The first run starts in the `bookstore` folder, like a chat does. The second run adds `--safe-mode`. Write one line: what will the second run not list?

2. **Run it in the project.**

```
claude -p "List your skills, MCP tools and rules." --output-format json
```

   The answer is JSON. Read the `result` field. Write down `total_cost_usd`.

3. **Run it again with `--safe-mode`.**

```
claude --safe-mode -p "List your skills, MCP tools and rules." --output-format json
```

   Read `result` and `total_cost_usd` again. Compare both with step 2.

Why: without `--safe-mode`, a headless run loads everything in the folder: `CLAUDE.md`, the course skills, the MCP server from session 4, and the hooks from session 3. With `--safe-mode`, it loads nothing from the folder. It loads only what you name on the command line, and here that is nothing.

Done when: your notes say in one sentence what the second run did not list, and give the cost of both runs.

2. The review gate

The gate is the script `headless/gate.py`. It sends your newest commit to Claude, and stops on any verdict but `pass`.

1. **Write the answer shape.** In `headless/gate.py`, replace `# (I write this)` under `SCHEMA` with the lines below, inside the braces. Then write down which line allows only two verdicts, and which line makes both fields required.

```
"type": "object",
"required": ["verdict", "findings"],
"properties": {
    "verdict": {"enum": ["pass", "block"]},
    "findings": {"type": "array", "items": {"type": "string"}},
},
```

2. **Add the team rules.** Read `headless/team-rules.md`. In `gate.py`, replace `# (I write this)` under `RULES` with two strings. The first is the flag that adds a file to Claude's instructions (slide *The Gate: A Review Before Every Push*). The shape is `"<the flag>", "headless/team-rules.md",`.

3. **Commit your gate.**

```
git add -A .
git commit -m "Add the review gate"
```

4. **Run the gate on a clean change.** Add the line `Gate test.` at the end of `README.md`. Then:

```
git commit -am "Gate test"
uv run headless/gate.py
```

5. **Predict.** Rule 1 says: a new endpoint answers an id that is not a number with 400. The next step adds an endpoint that answers with 500. Will the gate block it? Write *yes* or *no*.

6. **Let Claude break rule 1.** `--safe-mode` keeps your rule files from session 2 (in `.claude/rules/`) out. Those rules would make Claude answer 400. `acceptEdits` lets Claude edit files without asking.

```
claude --safe-mode -p "Do the task in headless/break.txt" --permission-mode acceptEdits
```

7. **Check the diff.** Run `git diff`. Note what the new endpoint answers for an id that is not a number.

8. **Commit, and run the gate twice.** Note if both runs give the same verdict and findings. They can differ: the verdict is Claude's judgment.

```
git add -A .
git commit -m "Add GET /api/reviews/{id}"
uv run headless/gate.py
uv run headless/gate.py
```

Why: Claude only gives a verdict. The script decides what happens, and a missing verdict counts as `block`.

Done when: the README change got `Verdict: pass`. The endpoint got `Verdict: block` with a finding on rule 1, or `pass` if Claude wrote 400.

3. The fix run

The gate blocked a commit. A second headless run can fix it, but a person still decides. If the gate passed the endpoint in task 2, skip this task.

1. **Read the command.** Open `headless/fix.py` and find the `claude` command. Write down its permission mode, and the one in `gate.py`. `dontAsk` means: Claude may not use a tool that would ask you. `acceptEdits` means: Claude may

edit files without asking. Why does the fix run get a different mode?

2. **Predict.** The fix run gets only the gate's verdict and findings. Claude fixes the code, and you commit it. Will the gate say `pass`? Write *yes* or *no*.

3. **Run the fix.** The script sends the gate's last answer to Claude. Claude edits the code. It does not commit.

```
uv run headless/fix.py
```

4. **Check the fix.** Run `git diff`. Does it change only what the finding asks for? You decide. If not, undo it with `git restore .` and run step 3 again.

5. **Commit, and run the gate.**

```
git commit -am "Fix the gate findings"
uv run headless/gate.py
```

   If it says `block` again, read the new finding and do steps 3 to 5 again. `fix.py` reads the newest gate answer.

Why: the gate only reads, so it cannot break anything. The fix run may edit files, so a person reads the diff before it is committed. This is the responder from the slides, on your laptop.

Done when: the gate gave `Verdict: pass` on your fix commit. Your notes say why the fix run may edit files and the gate may not.

4. The morning brief

This task produces two briefs: one for a night with errors, and one for a night where the log stops at 01:40.

1. **Write the prompt.** Create `headless/brief-prompt.txt`. The script `headless/brief.py` sends this file to Claude, with the night's log (an export from a logging platform). Ask for four things in your own words. The first: *Write at most 10 lines.* Then: the worst problem first; every number with the log line it came from; one line when nothing went wrong.

2. **Run it on the full night.**

```
uv run headless/brief.py headless/night-full.txt
```

The script prints the brief and saves it in `headless/out/`, next to the input Claude got. Does the brief name the endpoint with the errors, give the count, and quote a line? If not, improve your prompt and run again.

3. **Predict, then run it on the second night.** This log stops at 01:40, because the log file was replaced by a new one (log rotation). Will your brief say that the night is not complete? Write *yes* or *no*. Then run:

```
uv run headless/brief.py headless/night-cut.txt
```

4. **Improve one sentence.** If your brief does not say that the log stops at 01:40, add one sentence to your prompt. For example: ask for the time range the log covers. Then run the command of step 3 again.

Done when: the full-night brief names the endpoint with the errors, gives the count and quotes a log line. The second brief says which time range it covers.

5. Closing: check your work

1. **Start the check.** Close Claude if it is open. Start `claude` again, and run `/verify-exercise 5`. This course command came with the project. It reads your gate, the files `gate.py` and `fix.py` saved in `headless/out/`, and both briefs. First it asks if your second brief noticed that the log stops at 01:40. Answer in one line.
2. **Read the report.** Write down one line from it.

Done when: one line from the report is in your notes. Bring it to the closing round.

Bonus: the gate before every push

A git `pre-push` hook runs the gate before every `git push`.

1. Create the file `.git/hooks/pre-push`, with no extension, with these two lines:

```
#!/bin/sh
exec uv run headless/gate.py
```

2. On macOS or Linux, make it executable. On Windows, Git runs it without this step.

```
chmod +x .git/hooks/pre-push
```

3. Make a repository on your disk to push to, and push. Here `--bare` is a git option, not the Claude flag.

```
git init --bare ../gate-target.git
git push ../gate-target.git HEAD
```

The gate checks your last commit. If it says `pass`, the push goes through. If it says `block`, the push stops.

Back at work

Pick one check you do by hand before every push. Write it as a rule in the style of `headless/team-rules.md`, and run the gate on your own repository.

Appendix: the gate in GitHub Actions

`headless/claude-review.yml` is the same gate as a workflow that runs on every pull request. It posts what it finds as one comment on the pull request. Copy it to `.github/workflows/` in a repository of your own. Run `claude setup-token`: it prints a token for your Claude subscription. In GitHub, save it under *Settings → Secrets and variables → Actions* as `CLAUDE_CODE_OAUTH_TOKEN`. The workflow reads the team rules from the branch you merge into, not from the pull request.

Appendix: who runs it, and who pays

Where it runs	Logs in with	Who acted	Who pays
`claude -p` or `claude --safe-mode -p` on your laptop	your login	you	your plan
`claude --bare -p` in a script	an API key	the API key	API billing
GitHub Actions	a token from `claude setup-token`, or an API key	the Claude GitHub App	the token owner's plan, or API billing, plus GitHub Actions time
A cloud scheduled task	your claude.ai account	you	your plan

Closing round

The trainer asks the room. Have these answers ready:

- Why did the two runs in task 1 answer differently?
- In the gate, what does Claude decide, and what does the script decide?
- Why may the fix run edit files, while the gate may not?
- Did your brief notice that the log stops at 01:40? Which sentence in your prompt made the difference?
- The gate treats "no verdict" as "block". Why is that the safe choice?